Repartición de Roles y Arquitectura Técnica — Laboratorio 9

> **Curso:** Desarrollo de Software — Tema 4: EcmaScript
> **Laboratorio:** Laboratorio 9: JavaScript moderno y consumo asíncrono de la API
> **Paradigma:** 100% Vanilla ECMAScript Moderno (ESM Nativo), Async/Await, Fetch API, AbortController y Store Reactivo
> **Proyecto:** GuateGanado (Consumo de ProyectoGanadero_BackEnd en Laravel)
> **Meta del Equipo:** 100/100 en la Rúbrica de Evaluación (6 Personas)

1. Arquitectura Modular del Frontend (Screaming Architecture + ESM)

El código JavaScript se estructura en módulos con responsabilidades aisladas, sin variables globales y bajo el estándar oficial de ECMAScript:

```
ProyectoGanadero_Frontend/
├── HTML/
│   ├── dashboard.html
│   ├── animales.html
│   └── registro-animal.html
└── JavaScript/
    ├── core/                          # Núcleo reutilizable del sistema
    │   ├── api/
    │   │   ├── ApiClient.js           # Clase HTTP: encapsula baseUrl, Bearer Token y manejo uni
    │   │   └── endpoints.js           # Diccionario centralizado de endpoints REST
    │   ├── store/
    │   │   ├── Store.js               # Gestor de Estado Reactivo Singleton (State-driven UI)
    │   │   └── actions.js             # Mutaciones y despachadores asíncronos
    │   └── utils/
    │       ├── debounce.js            # Función debounce idiomática con temporizador
    │       └── storage.js             # Abstracción segura de localStorage/sessionStorage
    ├── features/                      # Módulos organizados por dominio de negocio
    │   ├── catalogo/
    │   │   ├── CatalogoView.js        # Módulo de vista para animales.html
    │   │   ├── animalService.js       # Consumo asíncrono de /api/v1/animales
    │   │   ├── components/
    │   │   │   ├── AnimalCard.js      # Template literal nativo de tarjeta de ganado
    │   │   │   ├── Pagination.js      # Renderizador y controlador de paginación
    │   │   │   └── ViewStates.js      # Renderizador de los 4 estados (Loading, Empty, Error+Ret
    │   │   └── searchHandler.js       # Debounce + AbortController para búsquedas
    │   ├── formulario/
    │   │   ├── FormularioView.js      # Módulo de vista para registro-animal.html (creación/edic
    │   │   ├── formValidator.js       # Mapeo en el DOM de errores 422 de Laravel por campo
    │   │   └── draftRecovery.js       # Guardado y recuperación de borrador ante 401
    │   └── dashboard/
    │       ├── DashboardView.js       # Módulo de vista para dashboard.html
    │       └── metricsService.js      # Consumo concurrente con Promise.allSettled()
    └── main.js                        # Punto de entrada / Router básico
```

2. Matriz de Roles (Alineada al 100% de la Rúbrica)

#	Integrante	Rol Oficial en el Laboratorio 9	Criterio de la Rúbrica Asociado	Puntaje
1	Persona 1	**Arquitecto de API Client & Manejo de Errores**	Cliente de la API y manejo de errores	**20 pts**
2	Persona 2	**Arquitecto de Estado Reactivo (Store Singleton)**	Organización en módulos y arquitectura ESM	**20 pts**
3	Persona 3	**Desarrollador de UI & 4 Estados de Interfaz**	Estados de la interfaz (Carga, Vacío, Error, Datos)	**20 pts**
4	Persona 4	**Especialista en Búsqueda Optimizada & Paginación**	Búsqueda con Debounce y Cancelación (AbortController)	**20 pts**
5	Persona 5	**Desarrollador de Formulario & Errores 422 (CRUD)**	Sincronización de datos y formulario	**20 pts**
6	Persona 6	**Integrador de Vistas, Sesión 401 & QA de Entregables**	Expiración de sesión y evidencia de entregables	**Entregables**

3. Asignación Exhaustiva de Tareas por Rol

Persona 1: Arquitecto de API Client & Manejo de Errores (20 pts)

- **Archivos:** `JavaScript/core/api/ApiClient.js` y `endpoints.js`.
- **Encapsulamiento:** Clase ES6+ con propiedades privadas (`#baseUrl`, `#token`). Configuración de cabeceras `Accept: application/json` y `Authorization: Bearer`.
- **Fetch API:** Métodos `get()`, `post()`, `put()`, `delete()` sin callbacks anidadas.
- **Manejo de Errores:** Clase personalizada `ApiException` que extrae código HTTP (401, 403, 404, 422, 500) y cuerpo JSON de Laravel.
- **Detección de 401:** Notificación centralizada ante sesión expirada.

Persona 2: Arquitecto de Estado Reactivo — Store Singleton (20 pts)

- **Archivos:** `JavaScript/core/store/Store.js` y `actions.js`.
- **Singleton ESM:** Instancia única en memoria gracias a la semántica de módulos de ECMAScript.
- **Patrón Observador:** `getState()` inmutable, `setState()` y `subscribe()` para notificar reactivamente a las vistas.
- **State-Driven UI:** Los datos dirigen la interfaz; ningún cambio de datos requiere recargar la página web.

Persona 3: Desarrollador de UI & 4 Estados de Interfaz (20 pts)

- **Archivos:** `JavaScript/features/catalogo/CatalogoView.js`, `components/ViewStates.js` y `AnimalCard.js`.

- **Estado de Carga (Loading):** Indicador visual / skeleton con accesibilidad (`aria-busy="true"`).
- **Estado Vacío (Empty):** Mensaje semántico cuando no hay registros tras aplicar filtros.
- **Estado de Error con Reintento:** Mensaje de fallo del servidor con botón interactivo `Reintentar` que vuelve a invocar la promesa.
- **Estado de Datos:** Renderizado de la cuadrícula con las tarjetas dinámicas de ganado.

Persona 4: Especialista en Búsqueda Optimizada & Paginación (20 pts)

- **Archivos:** `JavaScript/core/utils/debounce.js` , `searchHandler.js` y `Pagination.js` .
- **Debounce Funcional:** Retraso de ~300ms al escribir para optimizar peticiones al backend.
- **Cancelación con AbortController:** Al escribir un nuevo término o cambiar de filtro, aborta la petición en curso previa (`controller.abort()`) para evitar condiciones de carrera (*race conditions*).
- **Paginación Funcional:** Manejo de `current_page` , `last_page` y parámetros con `URLSearchParams` .

Persona 5: Desarrollador de Formulario & Errores 422 (20 pts)

- **Archivos:** `JavaScript/features/formulario/FormularioView.js` y `formValidator.js` .
- **CRUD Asíncrono:** Creación con `POST /api/v1/animales` y edición con `PUT /api/v1/animales/{id}` .
- **Mapeo de Errores 422:** Extrae el objeto `errores` devuelto por Laravel e inyecta el mensaje específico en cada campo correspondiente (`#error-numero_arete`), activando `aria-invalid="true"` .

Persona 6: Integrador de Vistas, Sesión 401 & QA de Entregables (Garantía de Nota)

- **Archivos:** `JavaScript/features/formulario/draftRecovery.js` , `dashboard/DashboardView.js` y carpeta `Docs/` .
- **Manejo de 401 sin pérdida de datos:** Guarda borrador en `sessionStorage` al detectar token expirado y lo restaura tras autenticarse nuevamente.
- **Consumo Concurrente:** Carga de métricas en Dashboard con `Promise.allSettled()` .
- **Entregables Oficiales:** Grabación de video / capturas de los 4 estados, errores 422 por campo y prueba de 401. Certificación de CERO callbacks anidadas (100% `async/await`).

4. Reglas Técnicas Obligatorias

- **1. CERO Callbacks Anidadas:** Flujo 100% asíncrono con `async/await` y bloques `try/catch/finally` .
- **2. Módulos ESM Nativos:** `import` y `export` sin librerías externas ni herramientas de empaquetado complejas para este laboratorio.
- **3. State-Driven DOM:** Ninguna mutación manual huérfana de HTML; todo fluye a través de componentes y el Store.
- **4. AbortController Activo:** Toda búsqueda asíncrona cancela la petición anterior antes de despachar una nueva.